

Звіт

до Лабораторної роботи №15

Синхронізація потоків у Java

Студент: Колісніченко Артем Олександрович

Група: КН-924в

Викладач: Савченко В. М.

Рік: 2026

1. Мета роботи

Метою лабораторної роботи є набуття практичних навичок вирішення проблем спільного доступу до змінюваних даних у багатопотокових програмах Java.

У роботі потрібно показати стан перегонів, визначити критичну секцію, реалізувати небезпечний варіант без синхронізації та коректний варіант із використанням `synchronized`.

2. Варіант роботи

Варіант 5 — лічильник подій

За умовою варіанта потрібно реалізувати спільний лічильник подій. Кілька потоків одночасно збільшують один спільний лічильник.

Для цього створено класи `SharedCounter`, `EventSource` і `CounterTask`.

3. Короткі теоретичні відомості

Спільний змінюваний стан виникає тоді, коли кілька потоків мають доступ до одного об'єкта або змінної, і хоча б один потік змінює цей стан.

Якщо доступ до такого ресурсу не контролювати, результат виконання може залежати від випадкового порядку перемикання потоків. Така ситуація називається станом перегонів.

Критична секція — це фрагмент коду, який працює зі спільним ресурсом і не повинен одночасно виконуватися кількома потоками.

У Java для захисту критичних секцій можна використовувати ключове слово `synchronized`. Воно забезпечує взаємне виключення: тільки один потік одночасно може виконувати синхронізований метод або блок коду з тим самим монітором.

4. Опис предметної області

У лабораторній роботі реалізовано модель лічильника подій.

Подіями можуть бути будь-які дії, які потрібно підрахувати: запити, натискання кнопок, повідомлення, звернення до системи або інші операції.

У цій роботі кілька потоків імітують незалежні джерела подій. Кожне джерело багато разів збільшує один спільний лічильник.

5. Класи проєкту

Клас	Призначення
SharedCounter	Інтерфейс спільного лічильника.
UnsafeSharedCounter	Небезпечна реалізація лічильника без синхронізації.
SafeSharedCounter	Синхронізована реалізація лічильника.
EventSource	Джерело подій, яке має назву та кількість подій.

CounterResult	Результат виконання сценарію.
CounterTask	Задача потоку, яка збільшує спільний лічильник.
CounterService	Сервіс для запуску послідовного та паралельного сценаріїв.
Main	Демонстраційна програма.

6. Опис спільного ресурсу

Спільним ресурсом у роботі є лічильник подій.

У класах лічильника використовується поле:

```
private int value;
```

Це поле є спільним змінюваним станом, тому що кілька потоків одночасно викликають метод `increment()` і змінюють його значення.

Очікуваний результат обчислюється за формулою:

кількість потоків * кількість операцій на потік

У демонстрації використано:

$$4 * 100000 = 400000$$

7. Небезпечний варіант без синхронізації

У класі `UnsafeSharedCounter` метод `increment()` не синхронізований.

Приклад логіки небезпечного збільшення:

```
int current = value;
```

```
value = current + 1;
```

Ця операція виглядає простою, але фактично складається з кількох дій:

1. прочитати поточне значення;

2. обчислити нове значення;
3. записати результат назад.

Якщо два потоки одночасно прочитають однакове значення, вони можуть обчислити однаковий новий результат і записати його назад. У такому випадку одне оновлення буде втрачено.

Саме це і демонструє стан перегонів.

8. Синхронізований варіант

У класі `SafeSharedCounter` доступ до спільного поля `value` захищено за допомогою `synchronized`.

Синхронізованими є методи:

```
public synchronized void increment()
```

```
public synchronized int getValue()
```

```
public synchronized void reset()
```

Завдяки цьому одночасно тільки один потік може виконувати критичну операцію над лічильником.

Інші потоки очікують, поки поточний потік завершить роботу із синхронізованим методом і звільнить монітор об'єкта.

9. Критична секція і вибір монітора

Критичною секцією є зміна значення лічильника у методі `increment()`.

Саме ця операція змінює спільний змінюваний стан, тому вона повинна виконуватися атомарно.

У класі `SafeSharedCounter` використано синхронізований метод. Для нестатичного синхронізованого методу монітором є поточний об'єкт `this`.

Це означає, що кожен об'єкт `SafeSharedCounter` має власний монітор, і потоки синхронізуються саме на цьому об'єкті.

10. UML-діаграми

Для проєкту було створено UML-діаграму класів і діаграму послідовності.

Діаграма класів показує структуру програми, а діаграма послідовності демонструє запуск послідовного, небезпечного паралельного і безпечного паралельного сценаріїв.

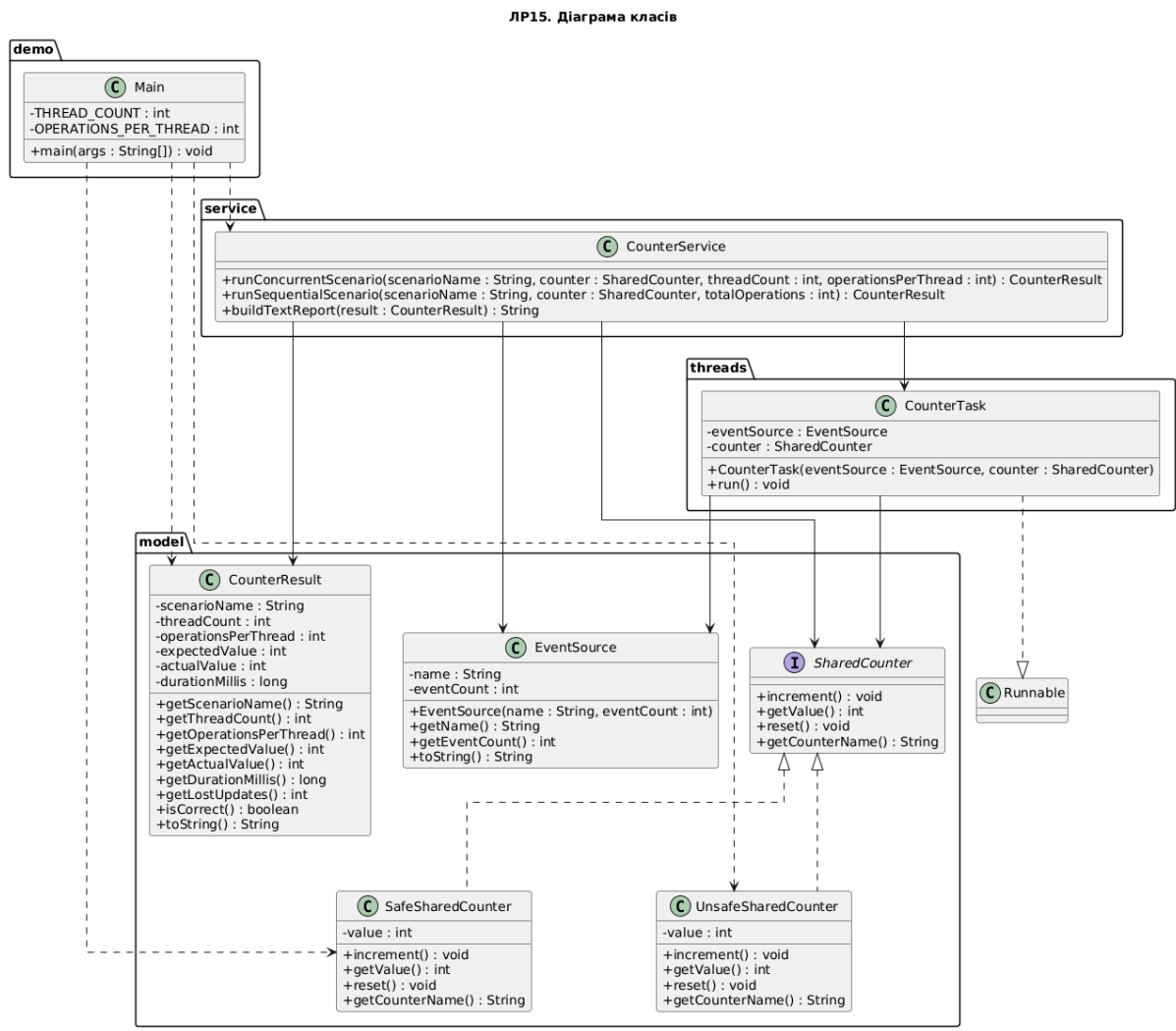


Рисунок 1 — UML-діаграма класів лабораторної роботи

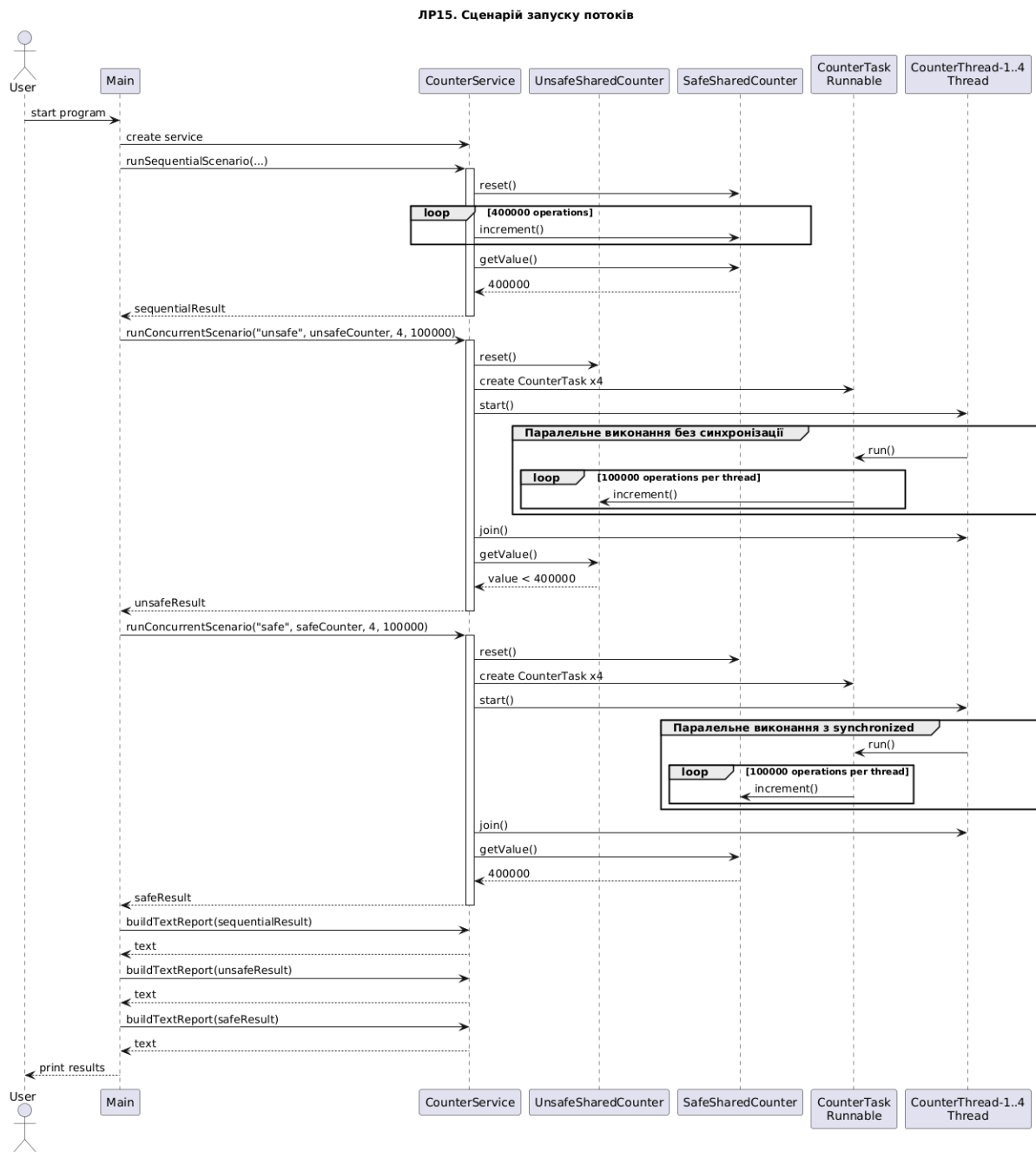


Рисунок 2 — Діаграма послідовності запуску сценаріїв

11. Параметри запуску експерименту

Параметр	Значення
----------	----------

Кількість потоків	4
Кількість операцій на потік	100000
Загальна кількість операцій	400000
Початкове значення лічильника	0
Очікуваний результат	400000

12. Результати виконання програми

У демонстраційній програмі виконуються три сценарії:

- послідовне виконання;
- паралельне виконання без синхронізації;
- паралельне виконання з synchronized.

```
Лабораторна робота №15
Варіант 5 — лічильник подій
Кількість потоків: 4
Операцій на потік: 100000
Очікуваний результат: 400000

Послідовне виконання
Потоків: 1
Операцій на потік: 400000
Очікуваний результат: 400000
Фактичний результат: 400000
Втрачено оновлень: 0
Час виконання: 5 мс
Результат коректний: так

CounterThread-2: початок обробки EventSource-2
CounterThread-4: початок обробки EventSource-4
CounterThread-1: початок обробки EventSource-1
CounterThread-3: початок обробки EventSource-3
CounterThread-4: завершено, подій: 100000
CounterThread-1: завершено, подій: 100000
CounterThread-2: завершено, подій: 100000
CounterThread-3: завершено, подій: 100000
Паралельне виконання без синхронізації
Потоків: 4
Операцій на потік: 100000
Очікуваний результат: 400000
Фактичний результат: 148326
Втрачено оновлень: 251674
Час виконання: 34 мс
Результат коректний: ні
```

```
CounterThread-1: початок обробки EventSource-1
CounterThread-2: початок обробки EventSource-2
CounterThread-3: початок обробки EventSource-3
CounterThread-4: початок обробки EventSource-4
CounterThread-2: завершено, подій: 100000
CounterThread-4: завершено, подій: 100000
CounterThread-1: завершено, подій: 100000
CounterThread-3: завершено, подій: 100000
Паралельне виконання з synchronized
Потоків: 4
Операцій на потік: 100000
Очікуваний результат: 400000
Фактичний результат: 400000
Втрачено оновлень: 0
Час виконання: 16 мс
Результат коректний: так
```

Рисунок 3 і 4 — Результат запуску демонстраційної програми

13. Порівняння часу виконання

Для вимірювання часу виконання використано метод `System.nanoTime()`.

Час вимірюється для послідовного і паралельних сценаріїв.

Паралельний сценарій не завжди є швидшим за послідовний, тому що синхронізація має накладні витрати: захоплення монітора, очікування, перемикання контексту та конкуренція потоків за доступ до критичної секції.

У синхронізованому варіанті результат є правильним, але час виконання може бути більшим, ніж у небезпечному варіанті без синхронізації.

14. Тестування

У роботі реалізовано модульні тести JUnit Jupiter.

Тестові класи:

- `SharedCounterTest`;
- `CounterServiceTest`.

Тести перевіряють:

- роботу синхронізованого лічильника;
- скидання значення лічильника;
- роботу небезпечного лічильника в одному потоці;
- створення джерела подій;
- послідовний сценарій;
- синхронізований паралельний сценарій;
- формування текстового звіту;
- перевірку неправильних параметрів.

Небезпечний багатопотоковий сценарій не використовується як строгий тест на помилку, тому що стан перегонів може проявлятися не в кожному запуску.

```
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.044 s -- in ua.khpi.oop.lab15.service.CounterServiceTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 11, Failures: 0, Errors: 0, Skipped: 0
```

Рисунок 5 — Результат виконання модульних тестів

15. Висновок

У ході виконання лабораторної роботи було реалізовано спільний лічильник подій для демонстрації синхронізації потоків у Java.

Було створено небезпечний лічильник `UnsafeSharedCounter`, у якому метод `increment()` не синхронізований. Через це кілька потоків можуть одночасно читати й змінювати одне поле `value`, що призводить до стану перегонів і втрати оновлень.

Також було створено безпечний лічильник `SafeSharedCounter`, у якому критичні операції захищені ключовим словом `synchronized`. Це гарантує, що тільки один потік одночасно змінює стан лічильника.

У результаті небезпечний варіант може показувати фактичне значення менше за очікуване, а синхронізований варіант повертає правильний результат.

Отже, використання `synchronized` дозволяє захистити критичну секцію, уникнути втрати оновлень і забезпечити коректний фінальний стан спільного ресурсу після завершення всіх потоків.